SIMATS ONLINE PROGRAMMING LAB ASSESSMENT CO1

Python Programming --- Milk Payment Calculator

TASK 1 – Understand the Problem and Identify the Data

Inputs:

- Farmer_id (text, because IDs may contain letters or leading zeros)
- Litres (int or Decimal, non-negative quantity)
- Fat_pct and snf_pct (fractional numeric percentages)
- Shift (text such as morning/evening).

Outputs:

- Rate_per_litre (money per litre), amount for one entry, and each farmer's ten-day total.

The program should also retain the entries if it must print an itemised record later.

Constants:

BASE_RATE = Decimal('30.00'),

FAT_STEP_VALUE = Decimal('0.40'),

SNF_STEP_VALUE = Decimal('0.30'),

STANDARD_FAT = Decimal('3.5'),

STANDARD_SNF = Decimal('8.5'),

STEP_SIZE = Decimal('0.1'),

MIN_FAT = Decimal('2.0'),

MAX_FAT = Decimal('7.0').

Types:

farmer_id and shift are str;

litres is best stored as Decimal for fractional litres or int if the analyser/scale guarantees whole litres;

fat and SNF are Decimal to preserve decimal readings; rate and amount are Decimal.

Counts such as number of entries are int; validation results are bool.

Rules:

Reject negative litres; require fat in the inclusive range 2.0–7.0; reject non-numeric inputs; use a documented rounding rule for 0.1 percentage-point steps; round money to two decimal places at the agreed transaction boundary; keep each farmer's ten-day total separate. Assumptions: the supplied rate card is authoritative; no different rate by shift is stated; no SNF range is specified; a zero-litre entry pays zero.

```
# Separate records scale better than hundreds of individually named variables.
entries = [
```

```
    {"farmer_id": "F001", "litres": Decimal("10"), "fat": Decimal("3.5"),
     "snf": Decimal("8.5"), "shift": "morning"}
]
```

For n entries, a loop processes each entry in O(n) time. A list of records uses O(n) space and is easier to extend, validate, sort, or audit. Three separate variables per farmer become 3 × 300 named values (plus IDs/shift), require repetitive code, and are hard to loop over. With one list/tuple per entry, the same loop handles 3 or 300 or 300,000 entries.

TASK 2 – Explore Values and Types in the Interpreter

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

The / operator performs true division; // floors the quotient (toward negative infinity), so -7 // 2 is -4 rather than -3; % gives the remainder consistent with floor division. The float equality is False because most decimal fractions, including 0.1 and 0.2, cannot be represented exactly in binary floating point.

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total
```

```
def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
```

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Expected:

float output is approximately 33400.30000000061; Decimal output is exactly 33400.30; the comparisons print False True. Decimal(amount) is correct when amount is a decimal string. Decimal(float(amount)) first imports the float's approximation, so the original decimal text cannot be recovered exactly and the sum may show extra digits.

Alternatives:

store integer paise (₹33.40 becomes 3340 paise) for exact addition when every amount has at most two decimal places; use Decimal for decimal arithmetic; round() can be useful for display or an explicitly defined rounding boundary, but repeatedly rounding binary floats does not make the underlying representation exact. For n payments, all methods are O(n) time and O(1) extra accumulation space. Use sys.getsizeof(value) to measure object sizes on your own Python build; sizes vary by Python version/platform.

TASK 3 – Choose Variables and Name the Constants

```
from decimal import Decimal


BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")
```

Use lowercase_with_underscores for variables and functions, descriptive names, and UPPER_CASE for constants. Python does not enforce constants; the uppercase style is a convention. Dynamic typing means a name can refer to different types over time: x = 10; x = 'ten' is legal, but later numeric operations may fail if x is a string. Multiple assignment assigns several names at once; a, b = b, a swaps values without a temporary variable.

Hard-coded 'magic numbers' repeated in k expressions require up to k edits for a rate revision and can cause inconsistent results. Named constants centralise the change. A RATE_CARD dictionary is useful if there are multiple seasons or rate cards, but its values should still be named and validated. Both constants and dictionary lookups are O(1); the dictionary uses a little extra memory, with no meaningful speed difference for this small program.

TASK 4 – Write Correct Expressions

```
from decimal import Decimal


STEP_SIZE = Decimal("0.1")
fat_steps = round((float(fat) - 3.5) / 0.1)
snf_steps = round((float(snf) - 8.5) / 0.1)
rate = round(30.00 + fat_steps * 0.40 + snf_steps * 0.30, 2)
amount = round(litres * rate, 2)
```

For financial code, prefer Decimal inputs and a documented rounding rule. With Decimal, compute step counts by dividing the difference by Decimal('0.1') and applying a chosen nearest-integer rule. Python's built-in round uses ties-to-even for exact halfway cases, so if the policy requires half-up, implement that policy explicitly.

```
>>>(4.2-3.5)/0.1
7.000000000000001
>>>round((4.2-3.5)/0.1)
7
>>>2**3**2
512
>>>-2**2
-4
>>>(-2)**2
4
```

The first surprise is binary floating-point approximation. round() gives the expected step count here. Exponentiation associates right-to-left, so 2 ** 3 ** 2 means 2 ** (3 ** 2). Exponentiation binds more tightly than unary minus, so -2

** 2 means -(2 ** 2), while (-2) ** 2 is 4. Another useful surprise: int(-3.8) is -3 (truncation toward zero), math.floor(-3.8) is -4, and round(2.5) is 2 because ties go to the even integer.

For fat = 4.2, the theoretical fat step count is 7; for fat = 3.1 it is -4. If float expressions are used, round() gives 7 and -4, while int() truncates the calculated quotient and floor() rounds down. For 3.1, floating representation may make the quotient close to -3.9999999999999964, so int() may produce -3 and floor() -4. Each is O(1) time and O(1) space; the difference is correctness at boundary values, not complexity.

Comparison chaining such as 2.0<= fat<= 7.0 means fat is at least 2.0 and at most 7.0. 'and' requires both conditions; 'or' requires at least one. Parentheses should be used when they make precedence clearer.

TASK 5 – Read Input and Convert Types

```
import decimal

from decimal import Decimal,InvalidOperation

    def read_decimal(prompt):
        """Read a valid decimal number, retrying after invalid input."""
        while True:
            text = input(prompt).strip()
            try:
                return Decimal(text)
            except InvalidOperation:
                print("Enter a number using a decimal point, e.g. 8.5.")


    farmer_id = input("Farmer ID: ").strip()
    litres = read_decimal("Litres: ")
    fat = read_decimal("Fat %: ")
    snf = read_decimal("SNF %: ")
    shift = input("Shift (morning/evening): ").strip().lower()
```

input() always returns str, so conversion is needed before arithmetic. '8.5' converts successfully; 'eight' and '8,5' are not accepted by Decimal in this format. Strip surrounding spaces first, so ' 8.5 ' is accepted. Use try/except rather than a simplistic string check because valid numeric formats can be more varied than a hand-written test.

```
    try:
        litres = Decimal(input("Litres: ").strip())
    except InvalidOperation:
        print("Invalid litres; enter a number such as 10 or 8.5.")
```

String-method prechecks can reject obvious mistakes but are incomplete; try/except directly tests whether conversion is possible and clearly handles 'eight' and '8,5'. Both approaches take O(1) time per input, excluding user think time. Format money with f-strings such as f'{amount:8.2f}' for width 8 and two digits after the decimal. Decimal

```python
import decimal
from decimal import Decimal ,InvalidOperation
def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Enter a number using a decimal point, e.g. 8.5.")

farmer_id = input("Farmer ID: ").strip()
litres = read_decimal("Litres: ")
fat = read_decimal("Fat %: ")
snf = read_decimal("SNF %: ")
shift = input("Shift (morning/evening): ").strip().lower()
try:
    litres = Decimal(input("Litres: ").strip())
except InvalidOperation:
    print("Invalid litres; enter a number such as 10 or 8.5.")
```

values can also be formatted with .2f.

```
Farmer ID: 52
Litres: 3
Fat %: 3
SNF %: 6
Shift (morning/evening): evening
Litres: 5
```

TASK 6 – Organise the Logic into Functions

from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

STANDARD_FAT = Decimal("3.5")

STANDARD_SNF = Decimal("8.5")

STEP_SIZE = Decimal("0.1")

CENT = Decimal("0.01")

def money(value):

"""Round a monetary value to cents."""

return value.quantize(CENT, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):

"""Calculate the milk rate per litre."""

fat_steps = int(

```
        ((fat - STANDARD_FAT) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
    )
    snf_steps = int(
        ((snf - STANDARD_SNF) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
    )
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return money(rate)
def payout(litres, fat, snf):
    """Calculate the payment for one milk entry."""
    litres = Decimal(str(litres))
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")
    return money(litres * rate_per_litre(fat, snf))
def ten_day_total(entries):
    """Calculate the total payment for ten days of entries."""
    total = Decimal("0.00")
```

```
for entry in entries:
        total += payout(
            entry["litres"],
            entry["fat"],
            entry["snf"]
        )
  return money(total)
# Example milk entries
entries = [
    {"litres": "10.0", "fat": "3.5", "snf": "8.5"},
    {"litres": "12.0", "fat": "4.0", "snf": "8.8"},
]
# Calculate and display results
for i, entry in enumerate(entries, start=1):
    rate = rate_per_litre(
        Decimal(entry["fat"]),
        Decimal(entry["snf"])
    )
    amount = payout(
        entry["litres"],
        entry["fat"],
        entry["snf"]
    )
 print(f"Entry {i}:")
    print(f"  Litres: {entry['litres']}")
    print(f"  Fat: {entry['fat']}")
    print(f"  SNF: {entry['snf']}")
    print(f"  Rate per litre: Rs. {rate:.2f}")
    print(f"  Amount: Rs. {amount:.2f}")
print("-" * 30)
```

print(f"Total payout: Rs. {ten_day_total(entries):.2f}")

A function should take required values as parameters and return a result. Printing is a side effect; returning lets other code test, reuse, or combine the result. Functions can read module constants, but should not reassign them. Local variables exist within a function; global/module-level names are defined outside functions. Docstrings explain the purpose. A single long script repeats logic and is harder to test; lambdas are best for short expressions, not multi-step validation. Function calls add small constant overhead, so overall processing remains O(n) for n entries.

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
CENT = Decimal("0.01")


def money(value):

    return value.quantize(CENT, rounding=ROUND_HALF_UP)


def rate_per_litre(fat, snf):

    fat_steps = int(
        ((fat - STANDARD_FAT) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
    )

    snf_steps = int(
        ((snf - STANDARD_SNF) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
```

```
    )

    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )

    return money(rate)


def payout(litres, fat, snf):

    litres = Decimal(str(litres))
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))

    if litres < 0:
        raise ValueError("Litres cannot be negative")

    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")

    if snf < 0:
        raise ValueError("SNF cannot be negative")

    return money(litres * rate_per_litre(fat, snf))

def ten_day_total(entries):
    """Calculate the total payment for ten days of entries."""
    total = Decimal("0.00")

    for entry in entries:
        total += payout(
            entry["litres"],
            entry["fat"],
            entry["snf"]
        )

    return money(total)

entries = [
    {"litres": "10.0", "fat": "3.5", "snf": "8.5"},
    {"litres": "12.0", "fat": "4.0", "snf": "8.8"},
]

for i, entry in enumerate(entries, start=1):
    rate = rate_per_litre(
        Decimal(entry["fat"]),
        Decimal(entry["snf"])
    )
    amount = payout(
        entry["litres"],
        entry["fat"],
        entry["snf"]
```

```
    print(f"Entry {i}:")
    print(f"  Litres: {entry['litres']}")
    print(f"  Fat: {entry['fat']}")
    print(f"  SNF: {entry['snf']}")
    print(f"  Rate per litre: Rs. {rate:.2f}")
    print(f"  Amount: Rs. {amount:.2f}")

print("-" * 30)
print(f"Total payout: Rs. {ten_day_total(entries):.2f}")
300 10019.999999999944 10020.00 difference -5.6E-11 float_s 1.8229999113827944e-
05 decimal_s 4.1129998862743 38e-05
6000 200399.99999997852 200400.00 difference -2.148E-8 float_s
0.00029372000062721 78 decimal_s 0.00062149000223143 96
150000 5010000.000000843 5010000.00 difference 8.43E-7 float_s
0.006666339999355841 decimal_s 0.012324179999268381
```

TASK 7 – Build a Module

Save the following as milkrate.py. Then save the second snippet as test_milkrate.py in the same folder.

```
"""Rate card and payout calculations for the milk co-operative."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
CENT = Decimal("0.01")

def _money(value):
    """Round a monetary value to cents using half-up rounding."""
    return value.quantize(CENT, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Return price per litre for the given fat and SNF readings."""
    fat_steps = int(((fat - STANDARD_FAT) / STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    snf_steps = int(((snf - STANDARD_SNF) / STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return _money(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE)

def payout(litres, fat, snf):
    """Return the rounded payment for one entry."""
    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not (Decimal("2.0") <= fat <= Decimal("7.0")):
        raise ValueError("Fat must be between 2.0 and 7.0")
```

```
    return _money(litres * rate_per_litre(fat, snf))

if __name__ == "__main__":
    for litres, fat, snf in [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3")),
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:>5} L  fat {fat}  snf {snf}  rate {rate:>6.2f}  pay {pay:>8.2f}")
```

```
# test_milkrate.py
from decimal import Decimal
import milkrate as mr

print(mr.rate_per_litre(Decimal("4.2"), Decimal("8.7")))
print(mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
```

import milkrate loads the module and refers to its names with milkrate.name; from milkrate import payout imports that function directly; import milkrate as mr gives it a short alias. The if __name__ == '__main__': block runs only when the file is executed directly. On import, __name__ is 'milkrate', so the demonstration table is not printed. Without this guard, importing the module would run the demonstration unexpectedly. Python searches the current folder and directories in sys.path. Importing shared functions avoids copying code; the module is loaded once per interpreter and cached in sys.modules.

```python
import decimal
from decimal import Decimal,ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
CENT = Decimal("0.01")

def _money(value):

    return value.quantize(CENT, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):

    fat_steps = int(((fat - STANDARD_FAT) / STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    snf_steps = int(((snf - STANDARD_SNF) / STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return _money(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE)

def payout(litres, fat, snf):

    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not (Decimal("2.0") <= fat <= Decimal("7.0")):
        raise ValueError("Fat must be between 2.0 and 7.0")
    return _money(litres * rate_per_litre(fat, snf))
if __name__ == "__main__":
    for litres, fat, snf in [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3")),
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:>5} L  fat {fat}  snf {snf}  rate {rate:>6.2f}  pay {pay:>8.2f}")

from decimal import Decimal
import milkrate as mr

print(mr.rate_per_litre(Decimal("4.2"), Decimal("8.7")))
print(mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
```

```
 10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
 12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

TASK 8 – Plan the Algorithm Before Coding

Algorithmic problem solving: (1) understand inputs/outputs and rules; (2) plan with pseudocode and a flowchart; (3) implement; (4) test normal, boundary, and invalid cases; (5) review results.

PSEUDOCODE:

INPUT farmer_id, litres, fat, snf, shift

```
IF any numeric input is invalid THEN
    REPORT error and ask for corrected input
ELSE IF litres < 0 THEN
    REPORT invalid quantity
ELSE IF fat < 2.0 OR fat > 7.0 THEN
    REPORT invalid fat
ELSE
    fat_steps <- nearest_integer((fat - 3.5) / 0.1)
    snf_steps <- nearest_integer((snf - 8.5) / 0.1)
    rate <- 30.00 + fat_steps * 0.40 + snf_steps * 0.30
    rate <- round_money(rate)
    amount <- round_money(litres * rate)
    SAVE entry and amount
    ten_day_total[farmer_id] <- ten_day_total[farmer_id] + amount
    DISPLAY rate, amount, ten_day_total[farmer_id]
END IF
```

Flowchart in words: Start → read fields → are all numeric fields valid? If no, show error and re-enter. If yes → litres ≥ 0? If no, show error. If yes → 2.0 ≤ fat ≤ 7.0? If no, show error. If yes → compute steps → rate → amount → update farmer's ten-day total → display/save result → End.

Hand trace (illustrative): standard entry 10 L, fat 3.5, SNF 8.5 gives 0 fat steps and 0 SNF steps, rate ₹30.00/L, amount ₹300.00. Entry 8.5 L, fat 4.2, SNF 8.7 gives +7 fat steps and +2 SNF steps, rate ₹30 + ₹2.80 + ₹0.60 = ₹33.40/L, amount ₹283.90. Entry 12 L, fat 3.1, SNF 8.3 gives -4 fat steps and -2 SNF steps, rate ₹30 - ₹1.60 - ₹0.60 = ₹27.80/L, amount ₹333.60. Total for these three entries = ₹917.50.

Storing every entry uses O(n) space and O(n) processing time. A running total uses O(1) extra space and O(n) time. Store entries when an audit trail, correction, reprint, sorting, or per-entry report is required; use only a running total when only the final total is needed.

TASK 9 – Implement and Test the Calculator

```
def payout(litres, fat, snf):
    if litres < 0:
        raise ValueError("Negative litres")
    if not D("2") <= fat <= D("7"):
        raise ValueError("Fat must be 2.0–7.0")
    if snf < 0:
        raise ValueError("Negative SNF")
    return (litres * rate_per_litre(fat, snf)).quantize(CENT, rounding=R)
tests = [
    ("standard", "10", "3.5", "8.5", "300.00"),
```

```
    ("fat/SNF above", "8.5", "4.2", "8.7", "283.90"),
    ("fat/SNF below", "12", "3.1", "8.3", "333.60"),
    ("minimum fat", "1", "2.0", "8.5", "24.00"),
    ("maximum fat", "1", "7.0", "8.5", "44.00"),
]
for name, l, f, s, expected in tests:
    actual = payout(D(l), D(f), D(s))
    print(name, "PASS" if actual == D(expected) else f"FAIL: {actual}")
try:
    payout(D("-1"), D("3.5"), D("8.5"))
    print("negative litres FAIL")
except ValueError:
    print("negative litres PASS")
```

Suggested test table: standard → expected ₹300.00; fat/SNF above → ₹283.90; fat/SNF below → ₹333.60; 1 L at minimum fat 2.0 with standard SNF → ₹24.00 (not ₹294.00); 1 L at maximum fat 7.0 with standard SNF → ₹44.00. The values in the code above for min/max should be ₹24.00 and ₹44.00 respectively; ensure your final test code uses these corrected values. Test non-numeric text at the input layer and assert that it is rejected; test fat below 2.0 and above 7.0; test negative litres. Test 5 entries by hand and compare with the program. Assertions rerun quickly after each code change; manual testing requires retyping and is more error-prone.

```python
from decimal import Decimal as D, ROUND_HALF_UP as R

BASE, FAT_STEP, SNF_STEP = D("30"), D("0.40"), D("0.30")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
CENT = D("0.01")


def rate_per_litre(fat, snf):
    f = int(((fat - STD_FAT) / STEP).quantize(D("1"), rounding=R))
    s = int(((snf - STD_SNF) / STEP).quantize(D("1"), rounding=R))
    return (BASE + f * FAT_STEP + s * SNF_STEP).quantize(CENT, rounding=R)


def payout(litres, fat, snf):
    if litres < 0:
        raise ValueError("Negative litres")
    if not D("2") <= fat <= D("7"):
        raise ValueError("Fat must be 2.0–7.0")
    if snf < 0:
        raise ValueError("Negative SNF")
    return (litres * rate_per_litre(fat, snf)).quantize(CENT, rounding=R)


tests = [
    ("standard", "10", "3.5", "8.5", "300.00"),
    ("fat/SNF above", "8.5", "4.2", "8.7", "283.90"),
    ("fat/SNF below", "12", "3.1", "8.3", "333.60"),
    ("minimum fat", "1", "2.0", "8.5", "24.00"),
    ("minimum fat", "1", "2.0", "8.5", "24.00"),
    ("maximum fat", "1", "7.0", "8.5", "44.00"),
]

for name, l, f, s, expected in tests:
    actual = payout(D(l), D(f), D(s))
    print(name, "PASS" if actual == D(expected) else f"FAIL: {actual}")

try:
    payout(D("-1"), D("3.5"), D("8.5"))
    print("negative litres FAIL")
except ValueError:
    print("negative litres PASS")
```

```
standard PASS
fat/SNF above PASS
fat/SNF below PASS
minimum fat PASS
maximum fat PASS
negative litres PASS
```

TASK 10 – Compare Expectations with Execution

```
from decimal import Decimal
from time import perf_counter

def float_sum(n):
    total = 0.0
    for _ in range(n):
        total += 33.40
    return total

def decimal_sum(n):
    total = Decimal("0.00")
    amount = Decimal("33.40")
    for _ in range(n):
        total += amount
```

```
    return total

for n in (300, 6000, 150000):
    t0 = perf_counter(); f = float_sum(n); tf = perf_counter() - t0
    t0 = perf_counter(); d = decimal_sum(n); td = perf_counter() - t0
    print(n, f, d, "difference", Decimal(str(f)) - d, "float_s", tf, "decimal_s", td)
```

Expected exact totals are 300 × 33.40 = ₹10,020.00; 6,000 × 33.40 = ₹200,400.00; 150,000 × 33.40 = ₹5,010,000.00. Float results may display tiny representation errors; Decimal gives exact decimal arithmetic when constructed from strings. Record your own perf_counter measurements because times depend on your computer, Python version, and current load.

For payments, calculate the amount for each transaction and round at the documented payment boundary; then sum those rounded transaction amounts using Decimal or integer paise. Rounding only the grand total can differ from summing individually rounded payouts. Use a dictionary cache only if the same fat/SNF pairs repeat often: lookup is average O(1), but it adds O(k) space for k distinct pairs. Benchmark before claiming it is faster; with only a few calculations, the cache may not matter.

```
from decimal import Decimal
from time import perf_counter

def float_sum(n):
    total = 0.0
    for _ in range(n):
        total += 33.40
    return total

def decimal_sum(n):
    total = Decimal("0.00")
    amount = Decimal("33.40")
    for _ in range(n):
        total += amount
    return total

for n in (300, 6000, 150000):
    t0 = perf_counter(); f = float_sum(n); tf = perf_counter() - t0
    t0 = perf_counter(); d = decimal_sum(n); td = perf_counter() - t0
    print(n, f, d, "difference", Decimal(str(f)) - d, "float_s", tf, "decimal_s", td)
```

```
300 10019.999999999944 10020.00 difference -5.6E-11 float_s 1.8229999113827944e-05 decimal_s 4.112999886274338e-05
6000 200399.99999997852 200400.00 difference -2.148E-8 float_s 0.0002937200006272178 decimal_s 0.0006214900022314396
150000 5010000.000000843 5010000.00 difference 8.43E-7 float_s 0.006666339999355841 decimal_s 0.012324179999268381
```

TASK 11 – Recommend and Defend the Final Design

Recommendation: use Decimal values constructed from strings (or integer paise for final monetary totals), centralise the rate card in one module, validate all inputs, use a documented rounding policy, and keep an auditable record of entries. Test functions should cover standard, high/low fat and SNF, allowed boundaries, invalid values, and ten-day totals.

Rate revision: change the named constants in one module or load a validated rate card from a configuration file. Limitations: the original prompt does not define an SNF validity range, shift-dependent pricing, duplicate-entry handling, database persistence, access control, or tax/bonus rules. Those requirements must be agreed with the co-operative before production use. For district-union scale, the calculation itself is O(n), but reliable operation also needs persistent storage, audit logs, concurrency controls, backup, authentication, and versioned rate cards.

```
# Optional rate card loaded from JSON:
#{
#   "base_rate": "30.00",
#   "fat_step_value": "0.40",
#   "snf_step_value": "0.30",
#   "standard_fat": "3.5",
#   "standard_snf": "8.5",
#   "step_size": "0.1"
#}
import json
from decimal import Decimal
# Rate card data
raw = json.loads("""
{
   "base_rate": "30.00",
   "fat_step_value": "0.40",
   "snf_step_value": "0.30"
}
""")
base_rate = Decimal(raw["base_rate"])
fat_step_value = Decimal(raw["fat_step_value"])
snf_step_value = Decimal(raw["snf_step_value"])
print("Base rate:", base_rate)
print("Fat step value:", fat_step_value)
print("SNF step value:", snf_step_value)
```

A JSON/text rate card makes updates easier without editing source code, but the file must be validated and version-controlled. Reading t settings at start-up takes O(t) time and O(t) space; for a small rate card this cost is negligible compared with the operational benefit. Report actual timing results from Task 10 as evidence rather than inventing them.

```python
import json
from decimal import Decimal

raw = json.loads("""
{
    "base_rate": "30.00",
    "fat_step_value": "0.40",
    "snf_step_value": "0.30"
}
""")

base_rate = Decimal(raw["base_rate"])
fat_step_value = Decimal(raw["fat_step_value"])
snf_step_value = Decimal(raw["snf_step_value"])

print("Base rate:", base_rate)
print("Fat step value:", fat_step_value)
print("SNF step value:", snf_step_value)
```

```
Base rate: 30.00
Fat step value: 0.40
SNF step value: 0.30
```

TASK 12 – Re-evaluate and Adapt the Design

Changed requirements: analyser readings have two decimal places; April–June uses base rate ₹32.00 and fat step value ₹0.45; the original SNF step value is assumed unchanged at ₹0.30 because no replacement is specified; a farmer supplying more than 500 L in ten days receives ₹0.50 per litre bonus. The exact threshold is strictly greater than 500, not greater than or equal to 500.

Two-decimal analyser readings make binary-float step calculations riskier. Use Decimal and calculate the nearest 0.1 percentage-point step using an explicitly chosen rounding rule. If the co-operative intends nearest-step rounding, readings such as 4.26% fat produce (4.26−3.50)/0.10 = 7.6 steps, rounded to 8; confirm this policy with the co-operative because it was not fully specified.

```
from decimal import Decimal as D, ROUND_HALF_UP as R

CENT, STEP = D("0.01"), D("0.1")

STANDARD_FAT, STANDARD_SNF = D("3.5"), D("8.5")

SNF_STEP, BONUS_LIMIT, BONUS_RATE = D("0.30"), D("500"), D("0.50")

RATE_CARDS = {
    "regular": {"base": D("30.00"), "fat_step": D("0.40")},
    "april_june": {"base": D("32.00"), "fat_step": D("0.45")},
}
def money(x):
    return x.quantize(CENT, rounding=R)
def steps(x, standard):
    return int(((x - standard) / STEP).quantize(D("1"), rounding=R))
def revised_rate(fat, snf, season):
    c = RATE_CARDS[season]
```

```
    return money(c["base"] + steps(fat, STANDARD_FAT) * c["fat_step"]
                 + steps(snf, STANDARD_SNF) * SNF_STEP)
def ten_day_payment(entries, season):
    litres = sum((e["litres"] for e in entries), D("0"))
    milk = sum((money(e["litres"] * revised_rate(
        e["fat"], e["snf"], season)) for e in entries), D("0.00"))
    bonus = money(litres * BONUS_RATE) if litres > BONUS_LIMIT else D("0.00")
    return litres, money(milk), bonus, money(milk + bonus)
entries = [
    {"litres": D("20"), "fat": D("3.8"), "snf": D("8.6")},
    {"litres": D("25"), "fat": D("4.0"), "snf": D("8.7")},
]
litres, milk, bonus, total = ten_day_payment(entries, "regular")
print("Total litres:", litres)
print("Milk payment: Rs.", milk)
print("Bonus: Rs.", bonus)
print("Grand total: Rs.", total)
# Worked examples:
# regular: 10 L, fat=4.20, snf=8.70 -> rate = 30 + 7*0.40 + 2*0.30 = 33.40
# April–June: same reading -> rate = 32 + 7*0.45 + 2*0.30 = 35.75
# 500 L exactly -> no bonus; 500.01 L -> bonus = 500.01 * 0.50 = 250.005,
# rounded to cents under the declared half-up policy -> ₹ 250.01.
```

Validation examples: regular season, 10 L at fat 4.20 and SNF 8.70 gives rate ₹33.40/L. April–June gives ₹35.75/L. If total litres are exactly 500.00, no bonus applies; at 500.01 L, the bonus is calculated because the condition is>500. The code calculates the bonus on total ten-day litres. If the policy means bonus only on litres above 500 rather than on all litres once the threshold is crossed, that rule must be clarified before final submission.

```python
from decimal import Decimal as D, ROUND_HALF_UP as R
CENT, STEP = D("0.01"), D("0.1")
STANDARD_FAT, STANDARD_SNF = D("3.5"), D("8.5")
SNF_STEP, BONUS_LIMIT, BONUS_RATE = D("0.30"), D("500"), D("0.50")
RATE_CARDS = {
    "regular": {"base": D("30.00"), "fat_step": D("0.40")},
    "april_june": {"base": D("32.00"), "fat_step": D("0.45")},
}
def money(x):
    return x.quantize(CENT, rounding=R)
def steps(x, standard):
    return int(((x - standard) / STEP).quantize(D("1"), rounding=R))
def revised_rate(fat, snf, season):
    c = RATE_CARDS[season]
    return money(c["base"] + steps(fat, STANDARD_FAT) * c["fat_step"]
                 + steps(snf, STANDARD_SNF) * SNF_STEP)
def ten_day_payment(entries, season):
    litres = sum((e["litres"] for e in entries), D("0"))
    milk = sum((money(e["litres"] * revised_rate(
        e["fat"], e["snf"], season)) for e in entries), D("0.00"))
    bonus = money(litres * BONUS_RATE) if litres > BONUS_LIMIT else D("0.00")
    return litres, money(milk), bonus, money(milk + bonus)
entries = [
    {"litres": D("20"), "fat": D("3.8"), "snf": D("8.6")},
    {"litres": D("25"), "fat": D("4.0"), "snf": D("8.7")},
]
litres, milk, bonus, total = ten_day_payment(entries, "regular")
print("Total litres:", litres)
print("Milk payment: Rs.", milk)
print("Bonus: Rs.", bonus)
print("Grand total: Rs.", total)
```

```
Total litres: 45
Milk payment: Rs. 1445.00
Bonus: Rs. 0.00
Grand total: Rs. 1445.00
```

Decision:

MODIFY the existing design rather than redesign it. Keep the functions and module, replace float readings with Decimal, pass the season explicitly to select a rate card, and add a ten-day bonus calculation. This reduces duplicated code and makes later seasonal changes easier. Test both seasons, the 500.00/500.01 boundary, two-decimal analyser readings, and fat/SNF values above and below standards.

Final submission checklist

- Replace any illustrative timing values with measurements from your own run.
- Run every code block;
- keep milkrate.py and test_milkrate.py in the same folder.
- Check the Task 9 boundary-test expected values:
- 1 L at fat 2.0, standard SNF is ₹24.00; 1 L at fat 7.0, standard SNF is ₹44.00.
- Add any required name/register details and follow your lecturer’s file-naming rules.